Manav Rachna International Institute of Research and Studies

School of Computer Applications

PRACTICAL FILE

of

DATA STRUCTURE & ALGORITHM

Master of Computer Applications (MCA) – Semester 1

Academic Session 2026–2027

SUBMITTED BY	
Name	**:** Roshan Kumar
Roll No.	**:** 26MCAP101N028
Course	**:** MCA, Semester 1
SUBMITTED TO	
Dr. Sachin Sharma Assistant Professor School of Computer Applications (MRIIRS)	

INDEX

Practical 1: Basic List Operations 2
Practical 2: Quick Sort and Merge Sort..... 7
Practical 3: Array-Based Stack, Queue, Circular Queue and Browser History..... 12
Practical 4: Postfix Expression Evaluation..... 23
Practical 5: Infix-to-Postfix Conversion..... 28
Practical 6: Singly Linked List Operations 33
Practical 7: Circular Linked List Operations..... 38
Practical 8: Stack, Queue and Circular Queue Using Linked Lists..... 43
Practical 9: Linked List Concatenation and Polynomial Addition 51
Practical 10: Doubly Linked List Operations..... 57

Practical 1: Basic List Operations

Aim

To write a menu-driven Python program that performs insertion, deletion and display operations on a list, and handles empty lists and invalid positions safely.

Algorithm

1. Create an empty list dataand display the menu.
2. Read the user's choice. If it is not an integer, report an error and read it again.
3. **Insert at end:** read a value and append it to data.
4. **Insert at position:** read the position and the value. If $0 \leq$ position $\leq$ length, insert the value at that position; otherwise report an invalid position.
5. **Delete by position:** if the list is empty, report it. Otherwise, if $0 \leq$ position $<$ length, remove the element at that position; else report an invalid position.
6. **Delete by value:** if the list is empty, report it. Otherwise remove the first occurrence of the value, or report that the value was not found.
7. **Display:** print the list and its size, or report that the list is empty.
8. Repeat steps 2 to 7 until the user chooses Exit.

Python Program

```
data = []

while True:
    print("\n----- LIST OPERATIONS -----")
    print("1. Insert at end")
    print("2. Insert at position")
    print("3. Delete by position")
    print("4. Delete by value")
    print("5. Display list")
    print("6. Exit")

    # Read and validate user's choice
    try:
        choice = int(input("Enter your choice: "))
    except ValueError:
        print("Error: Please enter an integer.")
        continue

    if choice == 1:
        value = input("Enter value: ")
        data.append(value)
        print("Value inserted successfully.")

    elif choice == 2:
        try:
            position = int(input("Enter position: "))
            value = input("Enter value: ")
```

```
if 0 <= position <= len(data):
                data.insert(position, value)
                print("Value inserted successfully.")
            else:
                print("Invalid position.")

        except ValueError:
            print("Error: Position must be an integer.")

    elif choice == 3:
        if len(data) == 0:
            print("List is empty.")
        else:
            try:
                position = int(input("Enter position: "))

                if 0 <= position < len(data):
                    removed = data.pop(position)
                    print("Deleted value:", removed)
                else:
                    print("Invalid position.")

            except ValueError:
                print("Error: Position must be an integer.")

    elif choice == 4:
        if len(data) == 0:
            print("List is empty.")
        else:
            value = input("Enter value to delete: ")

            if value in data:
                data.remove(value)
                print("Value deleted successfully.")
            else:
                print("Value not found.")

    elif choice == 5:
        if len(data) == 0:
            print("List is empty.")
        else:
            print("List:", data)
            print("Size:", len(data))

    elif choice == 6:
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Please choose 1 to 6."
```

```
----- LIST OPERATIONS -----
1. Insert at end
2. Insert at position
3. Delete by position
4. Delete by value
5. Display list
6. Exit
Enter your choice: 1
Enter value: 2
Value inserted successfully.

----- LIST OPERATIONS -----
1. Insert at end
2. Insert at position
3. Delete by position
4. Delete by value
5. Display list
6. Exit
Enter your choice: 3
Enter position: 0
Deleted value: 2
```

Practical 2: Quick Sort and Merge Sort

Aim

To write a Python program that sorts a list using Quick Sort and Merge Sort, implemented manually, and to compare their time complexities.

Algorithm

Quick Sort (in place)

1. If low < high, partition arr[low..high] and obtain the pivot position p.
2. **Partition:** take pivot = arr[high] and set i = low - 1. For j from low to high - 1, if arr[j] ≤ pivot, increase i by 1 and swap arr[i] with arr[j].
3. Swap arr[i + 1] with arr[high] and return i + 1 as p.
4. Recursively sort arr[low..p - 1] and arr[p + 1..high].

Merge Sort

1. If the list has one element or none, return a copy of it.
2. Split the list at the middle index into a left half and a right half and sort each half recursively.
3. **Merge:** with i = j = 0, repeatedly compare left[i] and right[j], append the smaller one to the result and advance its index, until one half is exhausted.
4. Append the remaining elements of the other half and return the merged list.

Main program

1. Read the numbers, copy the list, sort one copy with Quick Sort and sort the original with Merge Sort.
2. Display the original list and both sorted results, then measure the running time of both methods for a random list and for an already sorted list.

Python Program

```
import random
import time


# Quick Sort
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]
    left = []
    right = []
```

```
    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick_sort(left) + [pivot] + quick_sort(right)

# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]

    return result
```

```
# Main program
data = list(map(int, input("Enter numbers separated by spaces:
").split()))

print("Original list:", data)

start = time.perf_counter()
q = quick_sort(data)
q_time = time.perf_counter() - start

start = time.perf_counter()
m = merge_sort(data)
m_time = time.perf_counter() - start

print("Quick Sort:", q)
print("Merge Sort:", m)

print("Quick Sort time:", q_time)
print("Merge Sort time:", m_time)

# Compare on random and sorted lists
random_list = [random.randint(1, 1000) for _ in range(1000)]
sorted_list = list(range(1000))

for name, arr in [("Random List", random_list),
                  ("Sorted List", sorted_list)]:
    start = time.perf_counter()
    quick_sort(arr)
    qt = time.perf_counter() - start

    start = time.perf_counter()
    merge_sort(arr)
    mt = time.perf_counter() - start

    print("\n", name)
```

```
print("Quick Sort time:", qt)
print("Merge Sort time:", mt)
```

```
Enter numbers separated by spaces: 2
Original list: [2]
Quick Sort: [2]
Merge Sort: [2]
Quick Sort time: 7.079994247760624e-06
Merge Sort time: 1.5500045265071094e-06

 Random List
Quick Sort time: 0.001337749999947846
Merge Sort time: 0.001995489998080302
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 73, in <module>
  File "<main.py>", line 19, in quick_sort
  File "<main.py>", line 19, in quick_sort
  File "<main.py>", line 19, in quick_sort
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Practical 3: Array-Based Stack, Queue, Circular Queue and Browser History

Aim

To implement a stack, a queue, a circular queue and a browser history (using two stacks) with arrays.

Algorithm

A. Stack using an array (top = -1 initially)

1. **Push:** if top = capacity - 1, report Stack Overflow; otherwise increase top by 1 and store the item at data[top].
2. **Pop:** if top = -1, report Stack Underflow; otherwise read data[top], decrease top by 1 and return the item.
3. **Peek:** if top = -1, report Stack Underflow; otherwise return data[top].
4. **Display:** print data[top], data[top - 1], ..., data[0].

B. Queue using an array (front = 0, rear = -1 initially)

1. **Enqueue:** if rear = capacity - 1, report Queue Overflow; otherwise increase rear by 1 and store the item at data[rear].
2. **Dequeue:** if front > rear, report Queue Underflow; otherwise read data[front], increase front by 1 and return the item.
3. **Front element:** if front > rear, report Queue Underflow; otherwise return data[front].
4. **Display:** print data[front] to data[rear].

C. Circular queue using an array (front = 0, size = 0 initially)

1. **Enqueue:** if size = capacity, report Queue Overflow; otherwise store the item at index (front + size) mod capacity and increase size by 1.
2. **Dequeue:** if size = 0, report Queue Underflow; otherwise read data[front], set front = (front + 1) mod capacity, decrease size by 1 and return the item.
3. **Front element:** if size = 0, report Queue Underflow; otherwise return data[front].
4. **Display:** for i from 0 to size - 1, print data[(front + i) mod capacity].

D. Browser history using two stacks (current = None initially)

1. **Visit:** if a current page exists, push it onto the backward stack; set current to the new page and clear the forward stack.
2. **Back:** if the backward stack is empty, report that there is no previous page; otherwise push the current page onto the forward stack and pop the backward stack into current.
3. **Forward:** if the forward stack is empty, report that there is no forward page; otherwise push the current page onto the backward stack and pop the forward stack into current.
4. **Display:** print the current page (and the contents of both stacks).

Python Program

```
A. Stack using array

stack = [ ]
stack.append(10)
stack.append(20)
stack.append(30)

print("Stack:", stack)
print("Popped element:", stack.pop())
print("Top element:", stack[-1])
print("Stack after pop:", stack)

# B. Queue using array
queue = []
queue.append(5)
queue.append(15)
queue.append(25)

print("\nQueue:", queue)
print("Dequeued element:", queue.pop(0))
print("Front element:", queue[0])
print("Queue after deletion:", queue)

# C. Circular Queue using array
size = 3
cq = [None] * size
front = 0
count = 0

def enqueue(value):
    global count
    if count == size:
        print("Circular Queue is full")
    else:
        index = (front + count) % size
        cq[index] = value
        count += 1

def dequeue():
    global front, count
    if count == 0:
        print("Circular Queue is empty")
    else:
        value = cq[front]
        cq[front] = None
        front = (front + 1) % size
        count -= 1
        print("Removed:", value)

enqueue(100)
enqueue(200)
enqueue(300)
print("\nCircular Queue:", cq)
dequeue()
print("After deletion:", cq)
```

```
# D. Browser History using two stacks
back_stack = []
forward_stack = []
current = None

def visit(page):
    global current
    if current is not None:
        back_stack.append(current)
    current = page
    forward_stack.clear()

def go_back():
    global current
    if back_stack:
        forward_stack.append(current)
        current = back_stack.pop()
    else:
        print("No previous page")

def go_forward():
    global current
    if forward_stack:
        back_stack.append(current)
        current = forward_stack.pop()
    else:
        print("No next page")

visit("google.com")
visit("youtube.com")
visit("github.com")
print("\nCurrent page:", current)

go_back()
print("After Back:", current)

go_forward()
print("After Forward:", current)
```

```
Stack: [10, 20, 30]
Popped element: 30
Top element: 20
Stack after pop: [10, 20]

Queue: [5, 15, 25]
Dequeued element: 5
Front element: 15
Queue after deletion: [15, 25]

Circular Queue: [100, 200, 300]
Removed: 100
After deletion: [None, 200, 300]

Current page: github.com
After Back: youtube.com
After Forward: github.com
```

Practical 4: Postfix Expression Evaluation

Aim

To write a Python program that evaluates a postfix expression using a stack.

Algorithm

1. Create an empty stack and split the expression into tokens using whitespace. If there are no tokens, report an error.
2. Read the next token from left to right.
3. If the token is an integer, push it onto the stack.
4. If the token is an operator, check that the stack holds at least two values; otherwise report an invalid expression.
5. Pop the stack to obtain the right operand, then pop it again to obtain the left operand.
6. Apply the operator as left operator right. If the operator is / and the right operand is 0, report division by zero.
7. Push the result onto the stack.
8. If the token is neither an integer nor an operator, report an invalid token.
9. Repeat steps 2 to 8 for all tokens.
10. If exactly one value remains on the stack, display it as the result; otherwise report that the expression has too many operands.

Python Program

```
OPERATORS = {"+", "-", "*", "/", "^"}

class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.items:
            raise IndexError("Stack Underflow")
        return self.items.pop()

    def size(self):
        return len(self.items)


def is_integer(token):
    try:
        int(token)
        return True
    except ValueError:
        return False


def fmt(value):
    if isinstance(value, float) and value.is_integer():
        return str(int(value))
    return str(value)
```

```
def apply_operator(op, left, right):
    if op == "+":
        return left + right
    if op == "-":
        return left - right
    if op == "*":
        return left * right
    if op == "/":
        if right == 0:
            raise ZeroDivisionError("Division by zero.")
        return left / right
    if left == 0 and right < 0:
        raise ZeroDivisionError(
            "Zero cannot be raised to a negative power."
        )
    return left ** right


def evaluate_postfix(expression):
    tokens = expression.split()

    if not tokens:
        raise ValueError("Invalid expression: expression is empty.")

    stack = Stack()

    print(f"{'Token':<7}{'Action':<26}Stack")

    for token in tokens:
        if is_integer(token):
            stack.push(int(token))
            action = "Push operand"

        elif token in OPERATORS:
            if stack.size() < 2:
                raise ValueError(
                    f"Operator '{token}' needs two operands."
                )

            right = stack.pop()
            left = stack.pop()

            result = apply_operator(token, left, right)
            stack.push(result)

            action = (
                f"{fmt(left)} {token} {fmt(right)} = {fmt(result)}"
            )

        else:
            raise ValueError(f"Invalid token '{token}'.")

        shown = ", ".join(fmt(x) for x in stack.items)
        print(f"{token:<7}{action:<26}[{shown}]")

    if stack.size() != 1:
        raise ValueError("Invalid expression: too many operands.")

    return stack.pop()


def main():
    print("Enter a postfix expression with spaces between tokens.")
    print("Operators: + - * / ^")
    print("Type q to quit.")

    while True:
        expression = input("\nPostfix expression: ")

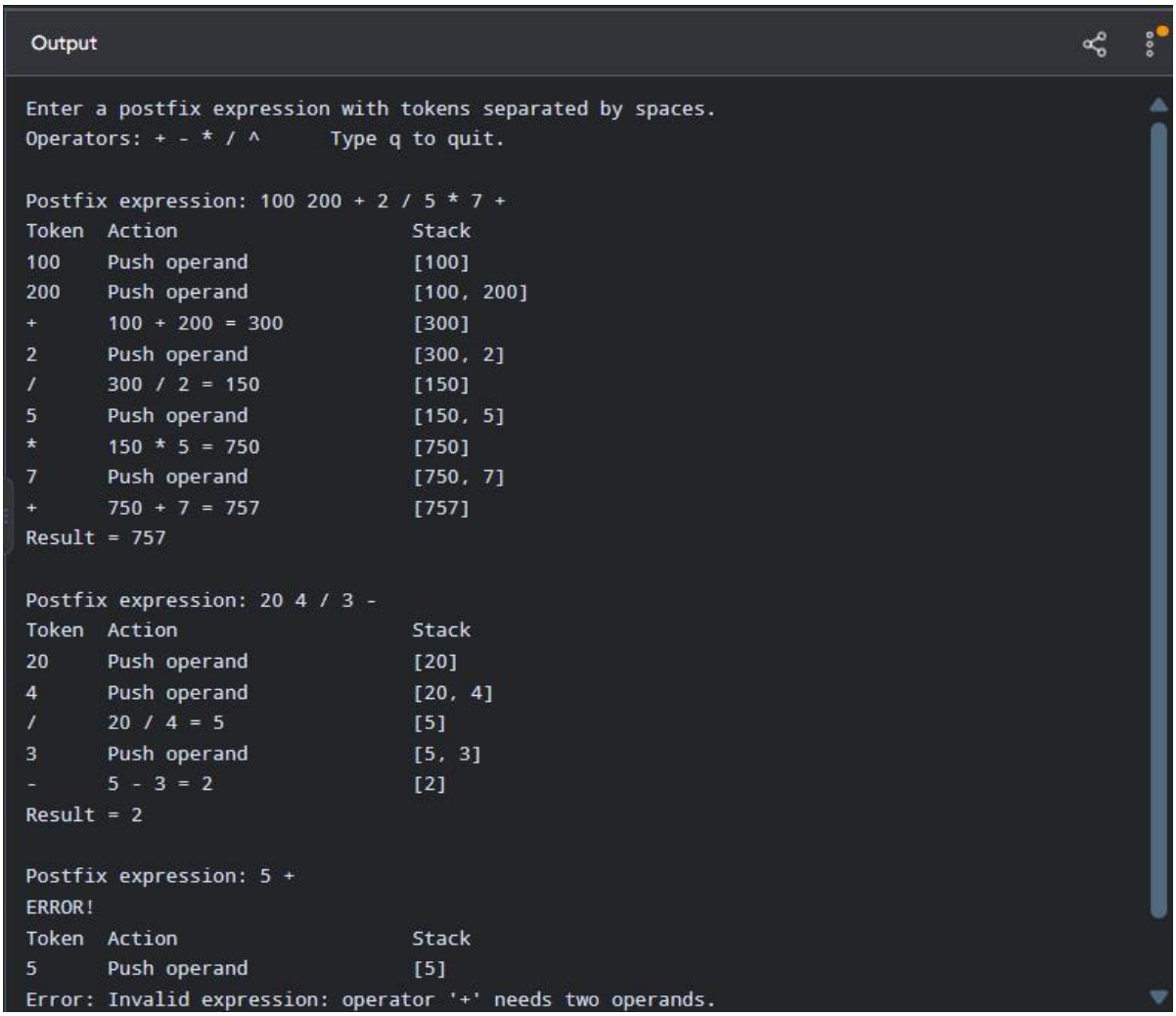
        if expression.strip().lower() == "q":
```

```
            print("Program ended.")
            break

        try:
            result = evaluate_postfix(expression)
            print("Result =", fmt(result))

        except (ValueError, ZeroDivisionError, OverflowError) as error:
            print("Error:", error)


if __name__ == "__main__":
    main(
```



```
Output

Enter a postfix expression with tokens separated by spaces.
Operators: + - * / ^      Type q to quit.

Postfix expression: 100 200 + 2 / 5 * 7 +
Token  Action                   Stack
100    Push operand             [100]
200    Push operand             [100, 200]
+      100 + 200 = 300          [300]
2      Push operand             [300, 2]
/      300 / 2 = 150            [150]
5      Push operand             [150, 5]
*      150 * 5 = 750            [750]
7      Push operand             [750, 7]
+      750 + 7 = 757            [757]
Result = 757

Postfix expression: 20 4 / 3 -
Token  Action                   Stack
20     Push operand             [20]
4      Push operand             [20, 4]
/      20 / 4 = 5               [5]
3      Push operand             [5, 3]
-      5 - 3 = 2                [2]
Result = 2

Postfix expression: 5 +
ERROR!
Token  Action                   Stack
5      Push operand             [5]
Error: Invalid expression: operator '+' needs two operands.
```

Practical 5: Infix-to-Postfix Conversion

Aim

To write a Python program that converts an infix expression into postfix notation using a stack.

Algorithm

1. Tokenize the expression and check that operands and operators are in a valid order. Create an empty operator stack and an empty output list.
2. Read the next token from left to right.
3. If the token is an operand, append it to the output.
4. If the token is (, push it onto the stack.
5. If the token is), pop operators from the stack to the output until (is found, and discard the (. If no (is found, report mismatched parentheses.
6. If the token is an operator o1, repeatedly pop the operator o2 at the top of the stack to the output while o2 is not (and either o2 has higher precedence than o1, or o2 has equal precedence and o1 is left-associative. Then push o1.
7. Repeat steps 2 to 6 for all tokens.
8. Pop all remaining operators from the stack to the output. If a (is found, report mismatched parentheses.
9. The output list, joined with spaces, is the postfix expression.

Python Program

```
PRECEDENCE = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}
RIGHT_ASSOCIATIVE = {"^"}
DIGITS = "0123456789"


class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[-1]

    def is_empty(self):
        return not self.items


def tokenize(expression):
    tokens = []
```

```
    number = ""

    for ch in expression:
        if ch in DIGITS:
            number += ch
            continue

        if number:
            tokens.append(number)
            number = ""

        if ch.isspace():
            continue

        if ch in PRECEDENCE or ch in "()":
            tokens.append(ch)
        else:
            raise ValueError(f"Invalid character '{ch}'.")

    if number:
        tokens.append(number)

    return tokens


def validate(tokens):
    if not tokens:
        raise ValueError("The expression is empty.")

    expect_operand = True

    for token in tokens:
        if token[0] in DIGITS:
            if not expect_operand:
                raise ValueError("Missing operator between
operands.")
            expect_operand = False

        elif token in PRECEDENCE:
            if expect_operand:
                raise ValueError(f"Operator '{token}' has no left
operand.")
            expect_operand = True

        elif token == "(":
            if not expect_operand:
                raise ValueError("Missing operator before '('.")

        else:
            if expect_operand:
                raise ValueError("Missing operand before ')'.")
            expect_operand = False

    if expect_operand:
        raise ValueError("The expression ends with an operator.")


def must_pop(top, current):
```

```
    if PRECEDENCE[top] > PRECEDENCE[current]:
        return True

    return (
        PRECEDENCE[top] == PRECEDENCE[current]
        and current not in RIGHT_ASSOCIATIVE
    )


def infix_to_postfix(tokens):
    stack = Stack()
    output = []

    print(f"{'Token':<7}{'Stack':<14}Output")

    for token in tokens:
        if token[0] in DIGITS:
            output.append(token)

        elif token == "(":
            stack.push(token)

        elif token == ")":
            while not stack.is_empty() and stack.peek() != "(":
                output.append(stack.pop())

            if stack.is_empty():
                raise ValueError("Mismatched parentheses.")

            stack.pop()

        else:
            while (
                not stack.is_empty()
                and stack.peek() != "("
                and must_pop(stack.peek(), token)
            ):
                output.append(stack.pop())

            stack.push(token)

        print(
            f"{token:<7}{' '.join(stack.items):<14}"
            f"{' '.join(output)}"
        )

    while not stack.is_empty():
        top = stack.pop()

        if top == "(":
            raise ValueError("Mismatched parentheses.")

        output.append(top)

    print(f"{'(end)':<7}{'':<14}{' '.join(output)}")

    return " ".join(output)
```

```
def main():
    print("Enter an infix expression using integers, operators and
parentheses.")
    print("Type q to quit.")

    while True:
        expression = input("\nInfix expression: ")

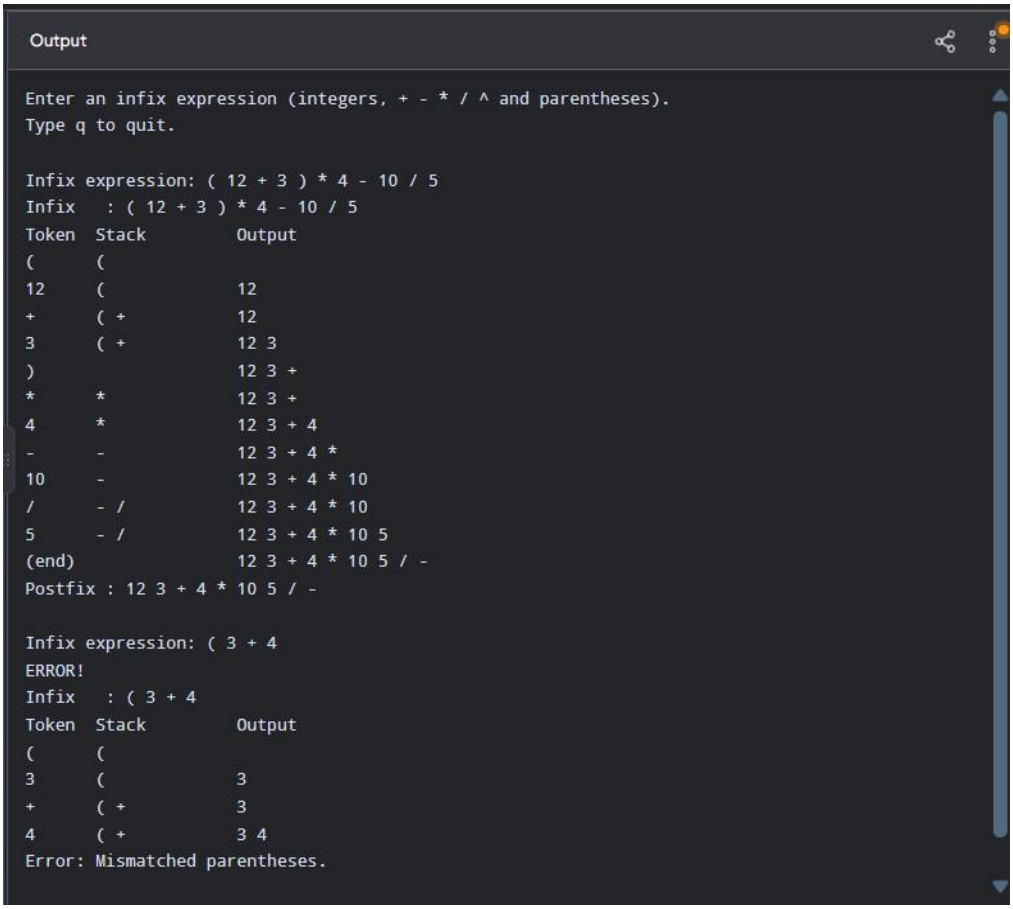
        if expression.strip().lower() == "q":
            print("Program ended.")
            break

        try:
            tokens = tokenize(expression)
            validate(tokens)

            print("Infix   :", " ".join(tokens))
            postfix = infix_to_postfix(tokens)
            print("Postfix :", postfix)

        except ValueError as error:
            print("Error:", error)


if __name__ == "__main__":
    main()
```


Output
Enter an infix expression (integers, + - * / ^ and parentheses).
Type q to quit.
Infix expression: (12 + 3) * 4 - 10 / 5
Infix : (12 + 3) * 4 - 10 / 5
Token Stack Output
((
12 (12
+ (+ 12
3 (+ 12 3
) 12 3 +
* * 12 3 +
4 * 12 3 + 4
- - 12 3 + 4 *
10 - 12 3 + 4 * 10
/ - / 12 3 + 4 * 10
5 - / 12 3 + 4 * 10 5
(end) 12 3 + 4 * 10 5 / -
Postfix : 12 3 + 4 * 10 5 / -
Infix expression: (3 + 4
ERROR!
Infix : (3 + 4
Token Stack Output
((
3 (3
+ (+ 3
4 (+ 3 4
Error: Mismatched parentheses.

Practical 6: Singly Linked List Operations

Aim

To write a Python program that performs insertion, deletion and display operations on a singly linked list.

Algorithm

1. Create an empty list with head = None.
2. **Insert at beginning:** create a new node, set its next to head and set head to the new node.
3. **Insert at end:** create a new node. If the list is empty, make it the head. Otherwise traverse to the last node (the node whose next is None) and set its next to the new node.
4. **Delete by value:** if the list is empty, report it. If the head holds the value, set head = head.next. Otherwise traverse with a reference previous; when previous.next holds the value, set previous.next = previous.next.next. If the end is reached, report that the value was not found.
5. **Display:** if the list is empty, report it; otherwise traverse from head, printing each value followed by an arrow, and finish with None.

Python Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def is_empty(self):
        return self.head is None

    def insert_at_beginning(self, data):
        node = Node(data)
        node.next = self.head
        self.head = node

    def insert_at_end(self, data):
        node = Node(data)

        if self.head is None:
            self.head = node
            return

        current = self.head

        while current.next is not None:
            current = current.next

        current.next = node
```

```
    def delete_value(self, value):
        if self.head is None:
            return False

        if self.head.data == value:
            self.head = self.head.next
            return True

        previous = self.head

        while previous.next is not None:
            if previous.next.data == value:
                previous.next = previous.next.next
                return True

            previous = previous.next

        return False

    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        current = self.head
        count = 0

        print("Head -> ", end="")

        while current is not None:
            print(current.data, end=" -> ")
            count += 1
            current = current.next

        print("None")
        print("Number of nodes:", count)


def main():
    lst = SinglyLinkedList()

    while True:
        print("\n1. Insert at beginning")
        print("2. Insert at end")
        print("3. Delete by value")
        print("4. Display")
        print("5. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            value = int(input("Enter value: "))
            lst.insert_at_beginning(value)
            print("Node inserted at beginning.")

        elif choice == "2":
```

```
            value = int(input("Enter value: "))
            lst.insert_at_end(value)
            print("Node inserted at end.")

        elif choice == "3":
            value = int(input("Enter value to delete: "))

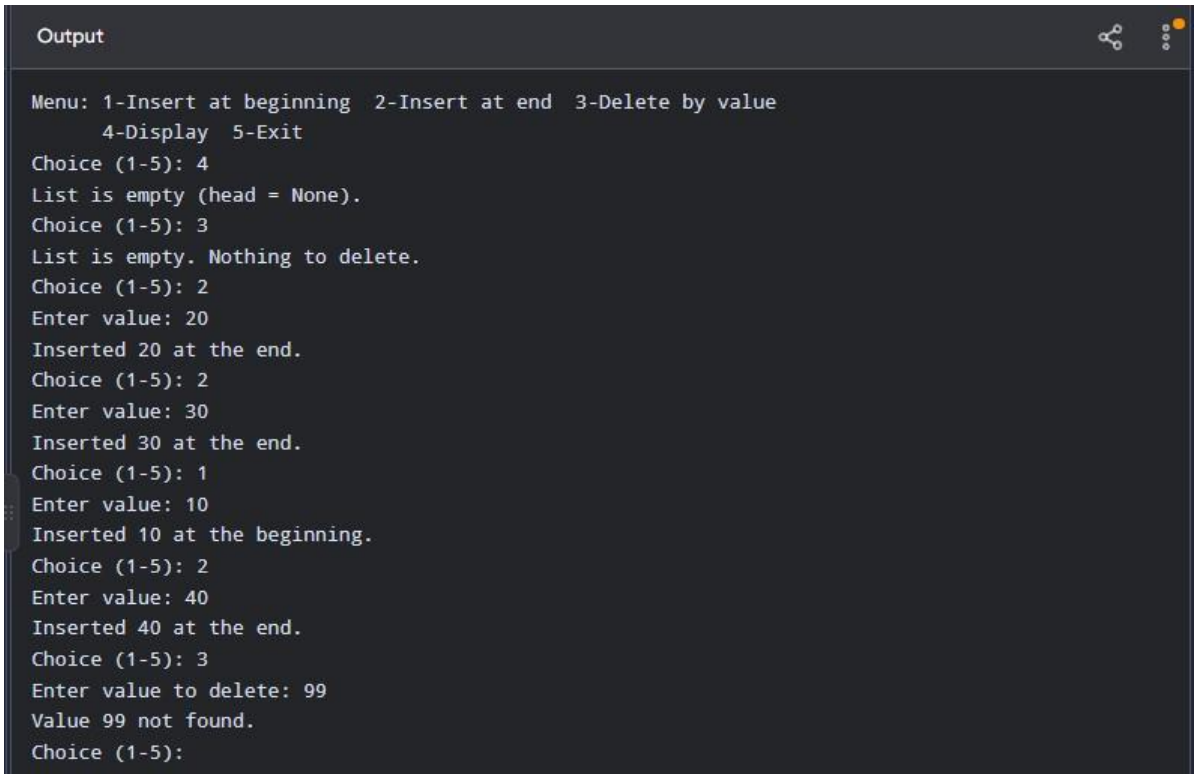
            if lst.delete_value(value):
                print("Node deleted successfully.")
            else:
                print("Value not found.")

        elif choice == "4":
            lst.display()

        elif choice == "5":
            print("Program ended.")
            break

        else:
            print("Invalid choice.")


if __name__ == "__main__":
    main()
```



Practical 7: Circular Linked List Operations

Aim

To write a Python program that performs insertion, deletion and display operations on a circular singly linked list.

Algorithm

1. Create an empty list with last = None.
2. **Insert at beginning:** create a node. If the list is empty, set node.next = node and last = node. Otherwise set node.next = last.next and last.next = node.
3. **Insert at end:** create a node. If the list is empty, set node.next = node. Otherwise set node.next = last.next and last.next = node. In both cases set last = node.
4. **Delete by value:** if the list is empty, report it. Start with previous = last and current = head. If current holds the value: if it is the only node, set last = None; otherwise set previous.next = current.next, and if current was last, set last = previous.
5. If the value is not found, move previous and current forward and continue. If current returns to the head, one full circle is complete and the value is reported as not found.
6. **Display:** if the list is empty, report it. Otherwise print each node from the head, stop when the head is reached again, and show that the last node points back to the head.

Python Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.last = None

    def is_empty(self):
        return self.last is None

    def insert_at_beginning(self, data):
        node = Node(data)
```

```
    if self.last is None:
        node.next = node
        self.last = node
    else:
        node.next = self.last.next
        self.last.next = node

def insert_at_end(self, data):
    node = Node(data)

    if self.last is None:
        node.next = node
    else:
        node.next = self.last.next
        self.last.next = node

    self.last = node

def delete_value(self, value):
    if self.last is None:
        return False

    head = self.last.next
    previous = self.last
    current = head

    while True:
        if current.data == value:
            if current == self.last and current == head:
                self.last = None
            else:
                previous.next = current.next

                if current == self.last:
                    self.last = previous
```

```
                return True

            previous = current
            current = current.next

            if current == head:
                return False

    def display(self):
        if self.last is None:
            print("List is empty.")
            return

        head = self.last.next
        current = head
        count = 0

        while True:
            print(current.data, end=" -> ")
            count += 1
            current = current.next

            if current == head:
                break

        print("(back to head)")
        print("Number of nodes:", count)


def main():
    lst = CircularLinkedList()

    while True:
        print("\n1. Insert at beginning")
```

```
print("2. Insert at end")
print("3. Delete by value")
print("4. Display")
print("5. Exit")

choice = input("Enter choice: ")

if choice == "1":
    value = int(input("Enter value: "))
    lst.insert_at_beginning(value)
    print("Node inserted at beginning.")

elif choice == "2":
    value = int(input("Enter value: "))
    lst.insert_at_end(value)
    print("Node inserted at end.")

elif choice == "3":
    if lst.is_empty():
        print("List is empty. Nothing to delete.")
        continue

    value = int(input("Enter value to delete: "))

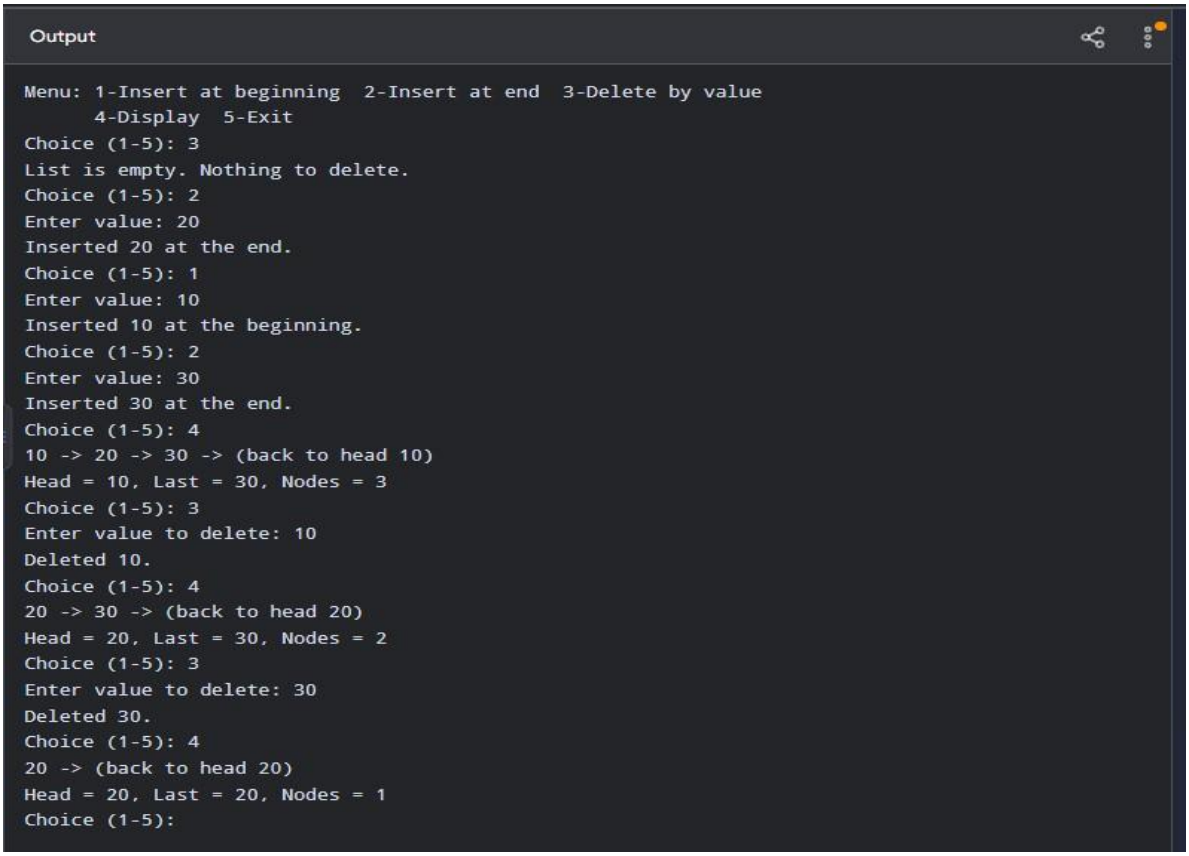
    if lst.delete_value(value):
        print("Node deleted successfully.")
    else:
        print("Value not found.")

elif choice == "4":
    lst.display()

elif choice == "5":
    print("Program ended.")
    break
```

```
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()
```



Practical 8: Stack, Queue and Circular Queue Using Linked Lists

Aim

To implement a stack, a queue and a circular queue using linked lists.

Algorithm

A. Stack using a linked list (top = None initially)

1. **Push:** create a node, set node.next = topand top = node.
2. **Pop:** if topis None, report Stack Underflow; otherwise read top.data, set top = top.next and return the item.
3. **Peek:** if topis None, report underflow; otherwise return top.data.
4. **Display:** traverse from topand print each node.

B. Queue using a linked list (front = rear = None initially)

1. **Enqueue:** create a node. If the queue is empty, set front = rear = node; otherwise set rear.next = nodeand rear = node.
2. **Dequeue:** if frontis None, report Queue Underflow. Otherwise read front.data and set front = front.next; if frontbecomes None, also set rear = None.
3. **Front:** if the queue is empty, report underflow; otherwise return front.data.
4. **Display:** traverse from frontto rearand print each node.

C. Circular queue using a linked list (front = rear = None initially)

1. **Enqueue:** create a node. If the queue is empty, set front = rear = node and node.next = node. Otherwise set node.next = front, rear.next = nodeand rear = node.
2. **Dequeue:** if the queue is empty, report underflow. If frontis rear(one node), set both to None. Otherwise set front = front.nextand rear.next = front.
3. **Front:** if the queue is empty, report underflow; otherwise return front.data.
4. **Display:** start at front, print each node and stop when frontis reached again.

Python Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedStack:
    def __init__(self):
        self.top = None

    def push(self, item):
        node = Node(item)
        node.next = self.top
        self.top = node

    def pop(self):
        if self.top is None:
            print("Stack is empty")
```

```
            return None

        item = self.top.data
        self.top = self.top.next
        return item

    def display(self):
        current = self.top
        print("Stack:", end=" ")

        while current:
            print(current.data, end=" ")
            current = current.next

        print()


class LinkedQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        node = Node(item)

        if self.rear is None:
            self.front = self.rear = node
        else:
            self.rear.next = node
            self.rear = node

    def dequeue(self):
        if self.front is None:
            print("Queue is empty")
            return None

        item = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        return item

    def display(self):
        current = self.front
        print("Queue:", end=" ")

        while current:
            print(current.data, end=" ")
            current = current.next

        print()


class LinkedCircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        node = Node(item)

        if self.front is None:
            self.front = self.rear = node
            node.next = node
        else:
            node.next = self.front
            self.rear.next = node
            self.rear = node
```

```
    def dequeue(self):
        if self.front is None:
            print("Circular queue is empty")
            return None

        item = self.front.data

        if self.front == self.rear:
            self.front = self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

        return item

    def display(self):
        if self.front is None:
            print("Circular queue is empty")
            return

        current = self.front
        print("Circular Queue:", end=" ")

        while True:
            print(current.data, end=" ")
            current = current.next

            if current == self.front:
                break

        print()


def main():
    stack = LinkedStack()
    queue = LinkedQueue()
    circular_queue = LinkedCircularQueue()

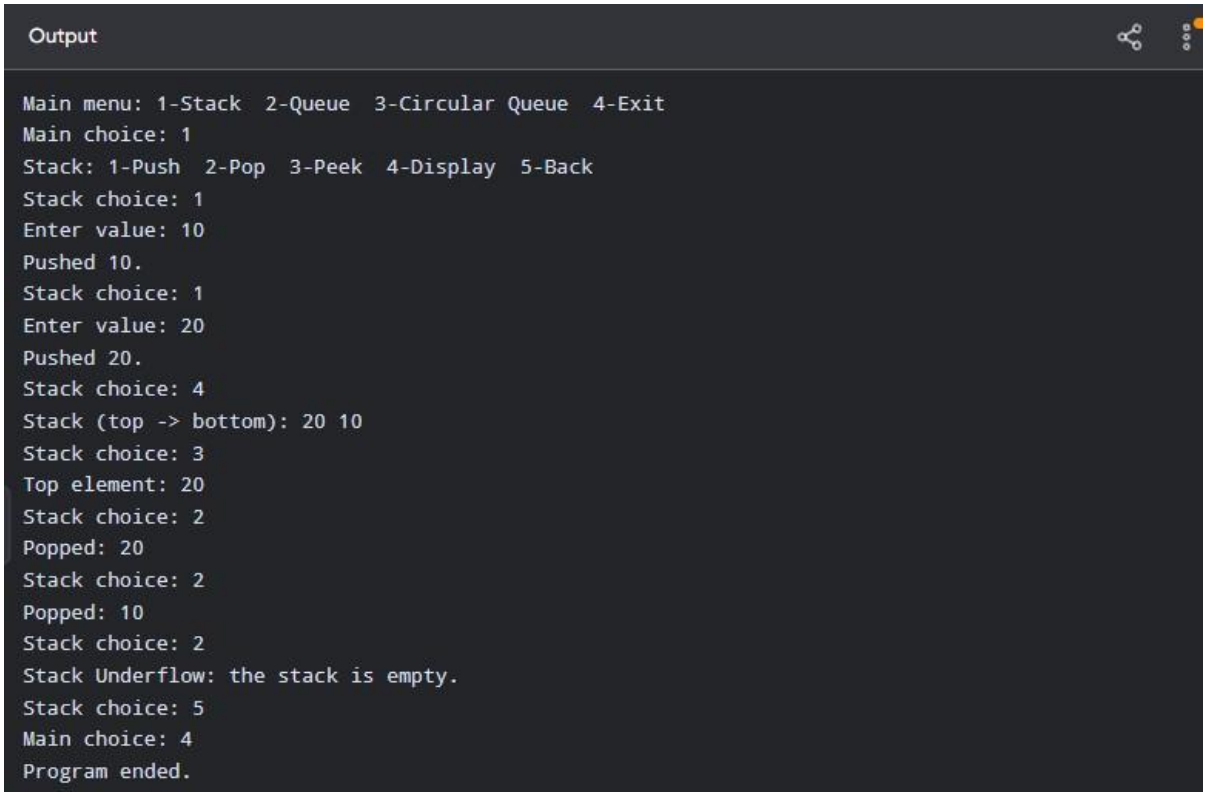
    # Stack operations
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.display()
    print("Popped:", stack.pop())
    stack.display()

    # Queue operations
    queue.enqueue(5)
    queue.enqueue(15)
    queue.enqueue(25)
    queue.display()
    print("Dequeued:", queue.dequeue())
    queue.display()

    # Circular queue operations
    circular_queue.enqueue(100)
    circular_queue.enqueue(200)
    circular_queue.enqueue(300)
    circular_queue.display()
    print("Dequeued:", circular_queue.dequeue())
    circular_queue.display()


if __name__ == "__main__":
    main()
```

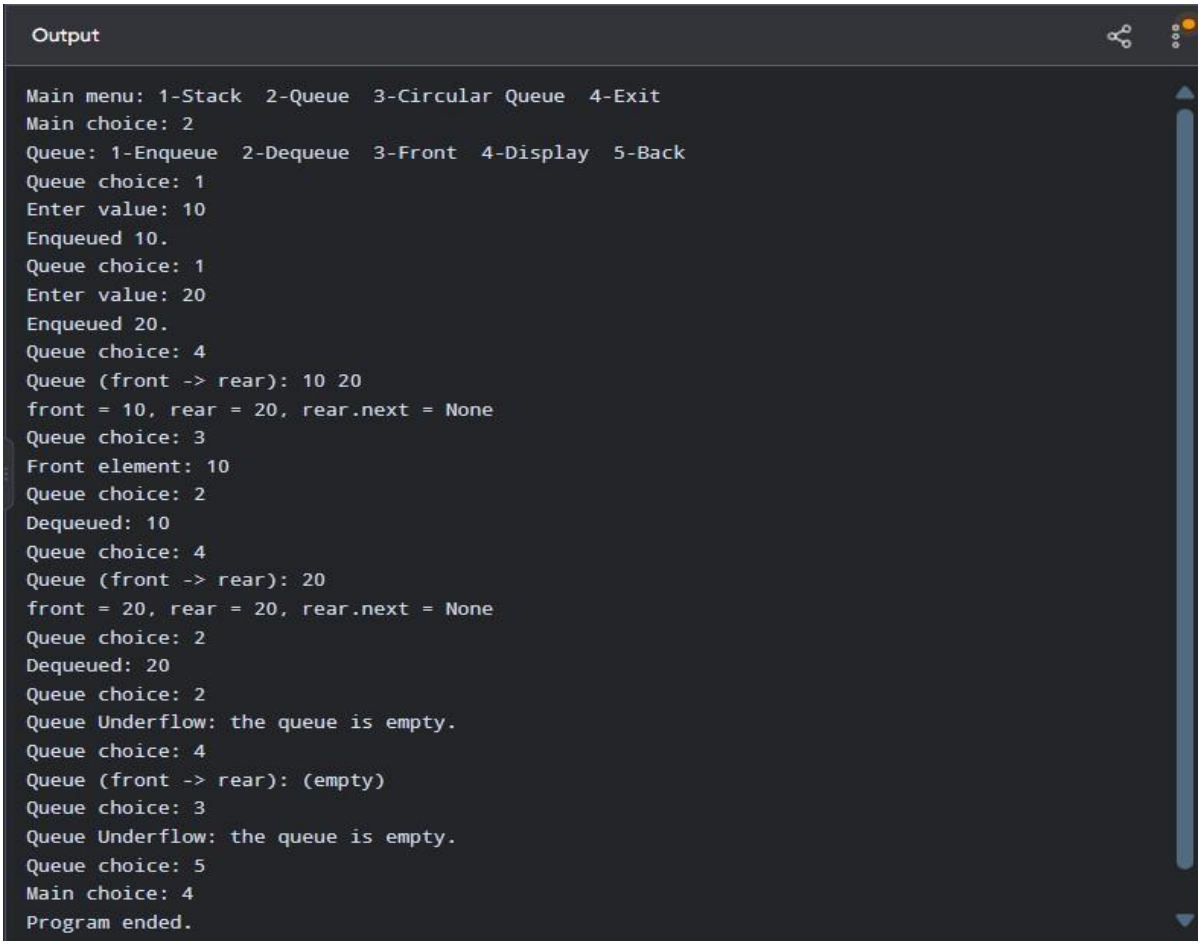
A. Stack



```
Output

Main menu: 1-Stack  2-Queue  3-Circular Queue  4-Exit
Main choice: 1
Stack: 1-Push  2-Pop  3-Peek  4-Display  5-Back
Stack choice: 1
Enter value: 10
Pushed 10.
Stack choice: 1
Enter value: 20
Pushed 20.
Stack choice: 4
Stack (top -> bottom): 20 10
Stack choice: 3
Top element: 20
Stack choice: 2
Popped: 20
Stack choice: 2
Popped: 10
Stack choice: 2
Stack Underflow: the stack is empty.
Stack choice: 5
Main choice: 4
Program ended.
```

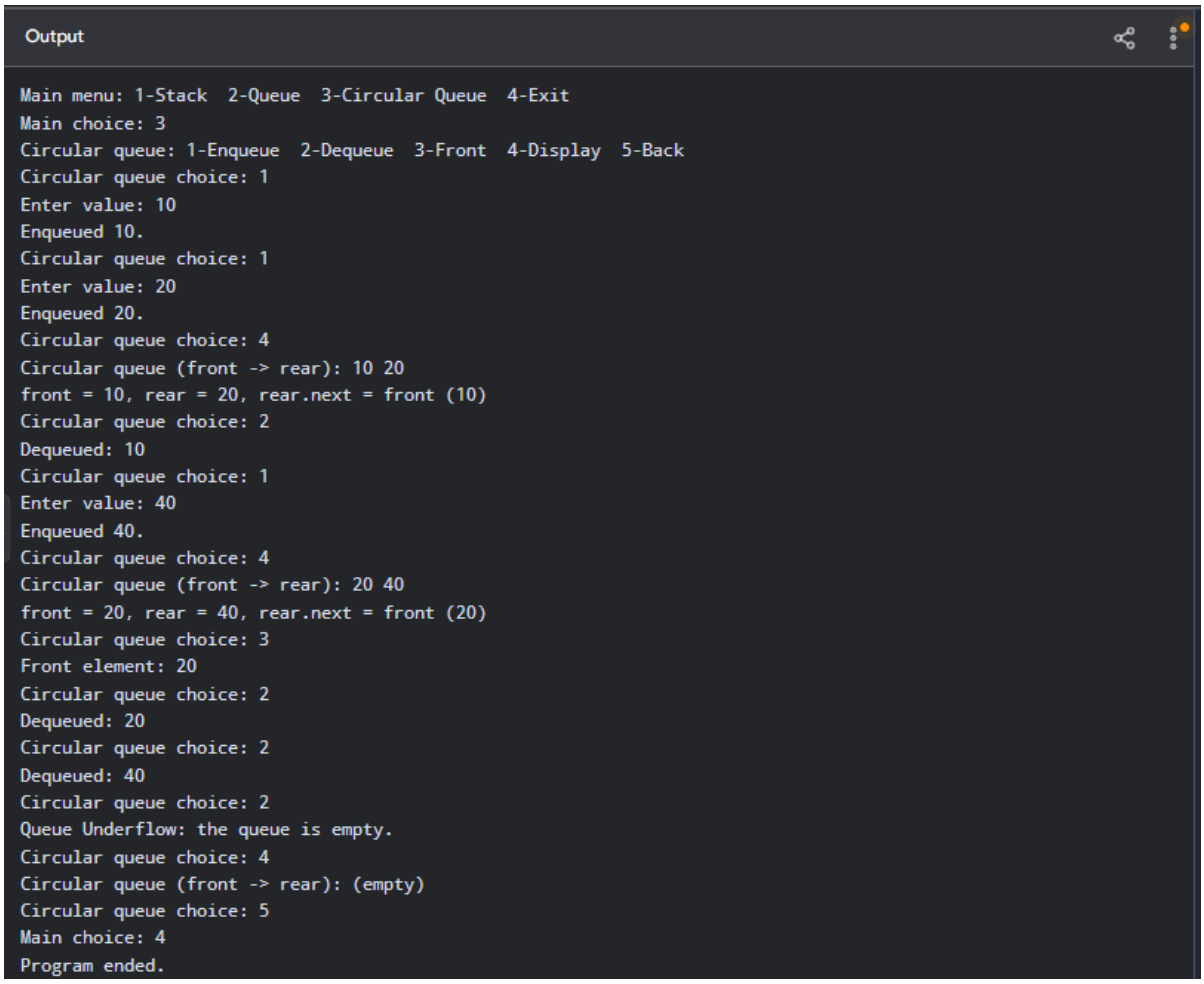
B. Queue



```
Output

Main menu: 1-Stack  2-Queue  3-Circular Queue  4-Exit
Main choice: 2
Queue: 1-Enqueue  2-Dequeue  3-Front  4-Display  5-Back
Queue choice: 1
Enter value: 10
Enqueued 10.
Queue choice: 1
Enter value: 20
Enqueued 20.
Queue choice: 4
Queue (front -> rear): 10 20
front = 10, rear = 20, rear.next = None
Queue choice: 3
Front element: 10
Queue choice: 2
Dequeued: 10
Queue choice: 4
Queue (front -> rear): 20
front = 20, rear = 20, rear.next = None
Queue choice: 2
Dequeued: 20
Queue choice: 2
Queue Underflow: the queue is empty.
Queue choice: 4
Queue (front -> rear): (empty)
Queue choice: 3
Queue Underflow: the queue is empty.
Queue choice: 5
Main choice: 4
Program ended.
```

C. Circular queue



```
Output

Main menu: 1-Stack  2-Queue  3-Circular Queue  4-Exit
Main choice: 3
Circular queue: 1-Enqueue  2-Dequeue  3-Front  4-Display  5-Back
Circular queue choice: 1
Enter value: 10
Enqueued 10.
Circular queue choice: 1
Enter value: 20
Enqueued 20.
Circular queue choice: 4
Circular queue (front -> rear): 10 20
front = 10, rear = 20, rear.next = front (10)
Circular queue choice: 2
Dequeued: 10
Circular queue choice: 1
Enter value: 40
Enqueued 40.
Circular queue choice: 4
Circular queue (front -> rear): 20 40
front = 20, rear = 40, rear.next = front (20)
Circular queue choice: 3
Front element: 20
Circular queue choice: 2
Dequeued: 20
Circular queue choice: 2
Dequeued: 40
Circular queue choice: 2
Queue Underflow: the queue is empty.
Circular queue choice: 4
Circular queue (front -> rear): (empty)
Circular queue choice: 5
Main choice: 4
Program ended.
```

Practical 9: Linked List Concatenation and Polynomial Addition

Aim

To write Python programs that concatenate two linked lists and that represent and add polynomials using linked lists.

Algorithm

A. Concatenation of two linked lists

1. Read the elements of the two lists and create two singly linked lists. Display both lists.
2. If list 2 is empty, stop: list 1 is unchanged.
3. If list 1 is empty, set head1 = head2.
4. Otherwise traverse list 1 to its last node and set last.next = head2.
5. Set head2 = None(the nodes now belong to list 1) and display the concatenated list.

B. Polynomial representation and addition

1. Read the terms of each polynomial as coeff exp pairs. For every pair with a non-zero coefficient, insert a node in descending order of exponent; if a node with the same exponent exists, add the coefficients and delete the node if the sum is zero.
2. Set pointers aand bto the first terms of the two polynomials.
3. While a or b is not None: if a.exp > b.exp copy the term of a and advance a; if b.exp > a.exp copy the term of b and advance b; if the exponents are equal, add the coefficients and advance both pointers.
4. Append a new node to the result for every term whose coefficient is not zero.
5. Display both input polynomials and the sum. An empty result is displayed as 0.

Python Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        node = Node(data)

        if self.head is None:
            self.head = node
            return

        current = self.head
        while current.next is not None:
            current = current.next
```

```
            current.next = node

        def concatenate(self, other):
            if other.head is None:
                return

            if self.head is None:
                self.head = other.head
            else:
                tail = self.head

                while tail.next is not None:
                    tail = tail.next

                tail.next = other.head

            other.head = None

        def to_text(self):
            if self.head is None:
                return "(empty list)"

            parts = []
            current = self.head

            while current is not None:
                parts.append(str(current.data))
                current = current.next

            return " -> ".join(parts) + " -> None"


    def read_list(label):
        while True:
            tokens = input(
                f"Elements of {label} (blank = empty): "
            ).split()

            try:
                values = [int(t) for t in tokens]
            except ValueError:
                print("Invalid input. Please enter whole numbers only.")
                continue

            lst = LinkedList()

            for value in values:
                lst.insert_at_end(value)

            return lst


    def concatenation_demo():
        first = read_list("list 1")
        second = read_list("list 2")
```

```
    print("List 1 :", first.to_text())
    print("List 2 :", second.to_text())

    first.concatenate(second)

    print("Result :", first.to_text())


# Part B: Polynomial Addition

class TermNode:
    def __init__(self, coeff, exp):
        self.coeff = coeff
        self.exp = exp
        self.next = None


class Polynomial:
    def __init__(self):
        self.head = None

    def add_term(self, coeff, exp):
        if coeff == 0:
            return

        if self.head is None or exp > self.head.exp:
            node = TermNode(coeff, exp)
            node.next = self.head
            self.head = node
            return

        previous = None
        current = self.head

        while current is not None and current.exp > exp:
            previous = current
            current = current.next

        if current is not None and current.exp == exp:
            current.coeff += coeff

            if current.coeff == 0:
                if previous is None:
                    self.head = current.next
                else:
                    previous.next = current.next
        else:
            node = TermNode(coeff, exp)
            node.next = current
            previous.next = node

    def add(self, other):
        result = Polynomial()
        tail = None

        a = self.head
        b = other.head
```

```
    while a is not None or b is not None:
        if b is None or (
            a is not None and a.exp > b.exp
        ):
            coeff, exp = a.coeff, a.exp
            a = a.next

        elif a is None or b.exp > a.exp:
            coeff, exp = b.coeff, b.exp
            b = b.next

        else:
            coeff = a.coeff + b.coeff
            exp = a.exp
            a = a.next
            b = b.next

        if coeff != 0:
            node = TermNode(coeff, exp)

            if tail is None:
                result.head = node
            else:
                tail.next = node

            tail = node

    return result

def to_text(self):
    if self.head is None:
        return "0"

    text = ""
    current = self.head

    while current is not None:
        magnitude = abs(current.coeff)

        if current.exp == 0:
            body = str(magnitude)
        else:
            body = "" if magnitude == 1 else str(magnitude)
            body += "x" + (
                "" if current.exp == 1
                else f"^{current.exp}"
            )

        if text == "":
            text = (
                ("-" if current.coeff < 0 else "") + body
            )
        else:
            text += (
                " - " if current.coeff < 0 else " + "
            ) + body
```

```
            current = current.next

        return text


def read_polynomial(label):
    while True:
        text = input(f"{label} (coeff exp, ...): ")
        poly = Polynomial()

        try:
            for part in text.split(","):
                fields = part.split()

                if len(fields) != 2:
                    raise ValueError

                coeff, exp = int(fields[0]), int(fields[1])

                if exp < 0:
                    raise ValueError

                poly.add_term(coeff, exp)

            return poly

        except ValueError:
            print("Invalid input. Example: 5 3, -4 2, 7 0")


def polynomial_demo():
    first = read_polynomial("Polynomial 1")
    second = read_polynomial("Polynomial 2")

    print("P1(x)         =", first.to_text())
    print("P2(x)         =", second.to_text())
    print("P1(x) + P2(x) =", first.add(second).to_text())

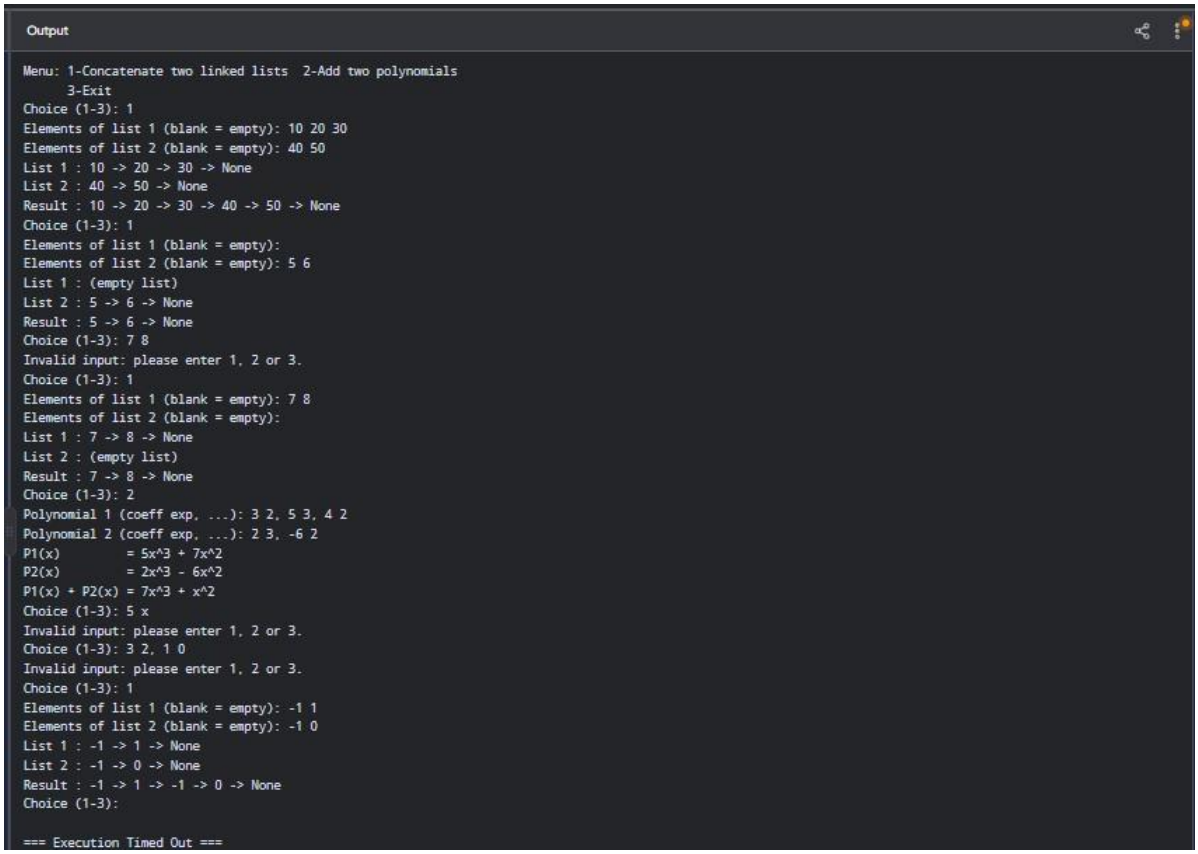

def main():
    print(
        "Menu: 1-Concatenate two linked lists  "
        "2-Add two polynomials"
    )
    print("      3-Exit")

    while True:
        try:
            choice = int(input("Choice (1-3): ").strip())
        except ValueError:
            print("Invalid input: please enter 1, 2 or 3.")
            continue

        if choice == 1:
            concatenation_demo()
        elif choice == 2:
```

```
            polynomial_demo()
        elif choice == 3:
            print("Program ended.")
            break
        else:
            print("Invalid choice. Please enter 1, 2 or 3.")


if __name__ == "__main__":
    main()
```



```
Output

Menu: 1-Concatenate two linked lists  2-Add two polynomials
      3-Exit
Choice (1-3): 1
Elements of list 1 (blank = empty): 10 20 30
Elements of list 2 (blank = empty): 40 50
List 1 : 10 -> 20 -> 30 -> None
List 2 : 40 -> 50 -> None
Result : 10 -> 20 -> 30 -> 40 -> 50 -> None
Choice (1-3): 1
Elements of list 1 (blank = empty):
Elements of list 2 (blank = empty): 5 6
List 1 : (empty list)
List 2 : 5 -> 6 -> None
Result : 5 -> 6 -> None
Choice (1-3): 7 8
Invalid input: please enter 1, 2 or 3.
Choice (1-3): 1
Elements of list 1 (blank = empty): 7 8
Elements of list 2 (blank = empty):
List 1 : 7 -> 8 -> None
List 2 : (empty list)
Result : 7 -> 8 -> None
Choice (1-3): 2
Polynomial 1 (coeff exp, ...): 3 2, 5 3, 4 2
Polynomial 2 (coeff exp, ...): 2 3, -6 2
P1(x)         = 5x^3 + 7x^2
P2(x)         = 2x^3 - 6x^2
P1(x) + P2(x) = 7x^3 + x^2
Choice (1-3): 5 x
Invalid input: please enter 1, 2 or 3.
Choice (1-3): 3 2, 1 0
Invalid input: please enter 1, 2 or 3.
Choice (1-3): 1
Elements of list 1 (blank = empty): -1 1
Elements of list 2 (blank = empty): -1 0
List 1 : -1 -> 1 -> None
List 2 : -1 -> 0 -> None
Result : -1 -> 1 -> -1 -> 0 -> None
Choice (1-3):

=== Execution Timed Out ===
```

Practical 10: Doubly Linked List Operations

Aim

To write a Python program that performs insertion, deletion and display operations on a doubly linked list.

Algorithm

1. Create an empty list with head = tail = None.
2. **Insert at beginning:** create a node. If the list is empty, set head = tail = node. Otherwise set node.next = head, head.prev = nodeand head = node.
3. **Insert at end:** create a node. If the list is empty, set head = tail = node. Otherwise set node.prev = tail, tail.next = nodeand tail = node.
4. **Delete by value:** traverse from headto find the node currentholding the value. If it is not found, report it.
5. If current.prev exists, set current.prev.next = current.next; otherwise currentis the first node, so set head = current.next.
6. If current.next exists, set current.next.prev = current.prev; otherwise currentis the last node, so set tail = current.prev.
7. **Display:** if the list is empty, report it. Otherwise traverse from headusing nextfor the forward display, and from tailusing prevfor the reverse display.

Python Program

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None

        self.prev = None




class DoublyLinkedList:

    def __init__(self):

        self.head = None

        self.tail = None


    def is_empty(self):
```

```
        return self.head is None

    def insert_at_beginning(self, data):
        node = Node(data)

        if self.head is None:
            self.head = self.tail = node
        else:
            node.next = self.head
            self.head.prev = node
            self.head = node

    def insert_at_end(self, data):
        node = Node(data)

        if self.tail is None:
            self.head = self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node

    def delete_value(self, value):
        current = self.head

        while current is not None and current.data != value:
            current = current.next
```

```
        if current is None:
            return False

        if current.prev is not None:
            current.prev.next = current.next
        else:
            self.head = current.next

        if current.next is not None:
            current.next.prev = current.prev
        else:
            self.tail = current.prev

        return True

    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        forward = []
        current = self.head

        while current is not None:
            forward.append(str(current.data))
            current = current.next

        backward = []
```

```
        current = self.tail

        while current is not None:
            backward.append(str(current.data))
            current = current.prev

        print("Forward :", " <-> ".join(forward))
        print("Reverse :", " <-> ".join(backward))
        print("Number of nodes:", len(forward))


def read_int(prompt):
    try:
        return int(input(prompt).strip())
    except ValueError:
        print("Invalid input: please enter a whole number.")
        return None


def main():
    lst = DoublyLinkedList()

    print("Menu: 1-Insert at beginning  2-Insert at end  3-Delete by
value")
    print("      4-Display  5-Exit")

    while True:
        choice = read_int("Choice (1-5): ")
```

```
if choice in (1, 2):
    value = read_int("Enter value: ")

    if value is None:
        continue

    if choice == 1:
        lst.insert_at_beginning(value)
        print(f"Inserted {value} at the beginning.")
    else:
        lst.insert_at_end(value)
        print(f"Inserted {value} at the end.")

elif choice == 3:
    if lst.is_empty():
        print("List is empty. Nothing to delete.")
        continue

    value = read_int("Enter value to delete: ")

    if value is not None:
        if lst.delete_value(value):
            print(f"Deleted {value}.")
        else:
            print(f"Value {value} not found.")

elif choice == 4:
```

```
            lst.display()


        elif choice == 5:
            print("Program ended.")
            break


        elif choice is not None:
            print("Invalid choice. Please enter a number from 1 to
5.")



if __name__ == "__main__":
    main()
```

